

Universidad Nacional del Altiplano

Facultad de Ingeniería Mecánica Eléctrica, Electrónica y Sistemas

Escuela Profesional de Ingeniería de Sistemas



Simulador Unificado de Planificación de CPU y Gestión de Memoria:

Implementación Modular y Evaluación Expositiva de Rendimiento

Autor: Abad Poma Maquera

Código de Matrícula: 241177

Curso: SIS215 – Sistemas Operativos

Docente: Ruelas Acero Donia Alizandra

Puno, Perú

2026

INTRODUCCIÓN

Los sistemas operativos gestionan dos recursos críticos que determinan el rendimiento de cualquier sistema de cómputo: el procesador y la memoria principal. Comprender su funcionamiento requiere observar los algoritmos en acción (Silberschatz et al., 2018). El Simulador del Núcleo S.O. responde a esa necesidad: es una aplicación web de página única (SPA) que ejecuta y visualiza, de forma sincronizada, la planificación de la CPU y la asignación de memoria, reportando diez métricas de rendimiento en tiempo real.

El simulador fue desarrollado íntegramente en JavaScript con módulos ES6, sin dependencias externas de *frameworks*. El usuario carga procesos en formato CSV o TXT, selecciona los algoritmos y observa en tiempo real el diagrama de Gantt, el mapa de memoria y las métricas actualizadas tick a tick (Tanenbaum & Bos, 2015).

Objetivo General

Desarrollar un simulador web interactivo que integre los principales algoritmos de planificación de CPU y gestión de memoria de un sistema operativo, para facilitar la comprensión práctica y visual de estos mecanismos en el marco del curso SIS215.

Objetivos Específicos

- Implementar FCFS, SJF, Round Robin y SRT con cálculo correcto de cinco métricas de CPU.
- Implementar First Fit, Best Fit, Worst Fit y Buddy System con seguimiento de cinco métricas de memoria.
- Diseñar una interfaz que muestre el diagrama de Gantt y el mapa de memoria de forma sincronizada con controles de animación.
- Generar tablas detalladas de planificación y gestión de memoria al finalizar cada simulación.

MARCO TEÓRICO

Planificación de CPU

La planificación de la CPU es el mecanismo por el cual el sistema operativo decide qué proceso listo ocupa el procesador en cada instante (Silberschatz et al., 2018). Los algoritmos se clasifican en *no expulsivos*, que no interrumpen al proceso en ejecución hasta que termine o se bloquee voluntariamente, y *expulsivos*, que pueden retirar la CPU cuando llega un proceso de mayor prioridad o se agota el quantum (Stallings, 2018).

FCFS – First Come, First Served. Atiende los procesos en el orden estricto de llegada. No es expulsivo. Su debilidad principal es el *efecto convoy*: un proceso de ráfaga larga retiene la CPU y fuerza la espera de todos los demás (Tanenbaum & Bos, 2015).

SJF – Shortest Job First. Selecciona el proceso con la menor ráfaga disponible. En su versión no expulsiva es óptimo en tiempo de espera promedio, pero requiere conocer las ráfagas de antemano (Silberschatz et al., 2018).

Round Robin. Asigna a cada proceso un *quantum* fijo. Si el proceso no termina, es expulsado y reingresa al final de la cola. Un quantum pequeño mejora la respuesta pero incrementa el overhead por cambios de contexto (Stallings, 2018).

SRT – Shortest Remaining Time. Versión expulsiva de SJF. Selecciona en cada tick el proceso con menor tiempo restante; si llega uno más corto, expulsa al actual. Minimiza el tiempo de espera promedio aunque puede provocar inanición (Tanenbaum & Bos, 2015).

Métricas de Planificación de CPU

El simulador calcula y exhibe cinco métricas de CPU en tiempo real (Silberschatz et al., 2018):

- **Total de procesos completados:** cantidad de procesos que han terminado su ejecución en el instante actual.

- **Uso de CPU (%):** fracción del tiempo en que la CPU no estuvo ociosa. $U_{CPU} = \frac{T_{total} - T_{idle}}{T_{total}} \times 100$.
- **Tiempo de espera promedio (T.E.):** promedio del tiempo acumulado en la cola de listos. $\overline{T_E} = \frac{1}{n} \sum_{i=1}^n (T_{inicio_i} - T_{llegada_i})$.
- **Tiempo de retorno promedio (T.R.):** promedio del tiempo total desde llegada hasta finalización. $\overline{T_R} = \frac{1}{n} \sum_{i=1}^n (T_{fin_i} - T_{llegada_i})$.
- **Throughput:** procesos completados por unidad de tiempo. $\theta = n/T_{total}$.

Gestión de Memoria

La memoria principal es un recurso limitado que el sistema operativo distribuye entre los procesos activos (Silberschatz et al., 2018). La **fragmentación externa** ocurre cuando el espacio libre total es suficiente pero está disperso en bloques discontinuos; la **fragmentación interna** ocurre cuando el proceso recibe más memoria de la que solicitó (Stallings, 2018).

First Fit. Asigna el proceso al primer bloque libre suficientemente grande. Es el más rápido en tiempo de búsqueda (Tanenbaum & Bos, 2015).

Best Fit. Busca el bloque libre más pequeño que sea suficiente. Minimiza el desperdicio inmediato pero genera residuos muy pequeños difíciles de reutilizar (Silberschatz et al., 2018).

Worst Fit. Asigna el proceso al bloque libre más grande. El residuo resultante es grande y potencialmente útil; en la práctica no supera a First Fit (Stallings, 2018).

Buddy System. Organiza la memoria en potencias de 2. Divide recursivamente el bloque más pequeño suficiente en dos gemelos (*buddies*) hasta alcanzar el tamaño requerido. Al liberar, fusiona gemelos libres hacia arriba, reduciendo la fragmentación externa a largo plazo (Knuth, 1973).

Métricas de Gestión de Memoria

El simulador reporta cinco métricas de memoria en tiempo real:

- **% Uso de memoria:** porcentaje del espacio disponible (excluido el SO) ocupado actualmente por procesos. $\%U_{mem} = \frac{\sum tamaño_asignado}{mem_disponible} \times 100$.
- **Memoria libre (KB):** espacio libre total en el instante actual; equivale a la fragmentación externa acumulada.
- **Fragmentación interna (KB):** suma del espacio no utilizado dentro de particiones asignadas. Con política dinámica es 0 KB; con Buddy System puede existir al redondear a potencias de 2. $FI = \sum (partición_i - solicitado_i)$.
- **Fragmentación externa (KB):** suma del espacio libre disperso. Coincide con la memoria libre en esquemas de asignación contigua dinámica.
- **Asignados / Total:** procesos que obtuvieron memoria sobre el total de solicitudes. Un valor menor que el total indica rechazos por falta de bloque contiguo suficiente.

DESCRIPCIÓN DEL SISTEMA

Arquitectura General

El sistema sigue una arquitectura modular de tres capas claramente separadas. La capa de **lógica de negocio** contiene los ocho algoritmos en módulos JavaScript independientes. La capa de **interfaz de usuario** agrupa los cuatro renderizadores (Gantt, mapa de memoria, métricas y log de pasos). La capa de **orquestración** (`main.js`) gestiona el ciclo de vida completo de la simulación, la línea de tiempo unificada y la comunicación entre las otras dos capas. La Tabla 1 resume los módulos.

Tabla 1

Estructura modular del Simulador del Núcleo S.O.

Archivo	Responsabilidad
<code>main.js</code>	Orquestador principal: estado global, ciclo tick a tick, animación
<code>parser.js</code>	Parseo de CSV/TXT, validación de campos y tabla editable
<code>planificadores/algoritmos.js</code>	Implementación de FCFS, SJF, Round Robin y SRT
<code>memoria/algoritmos.js</code>	Implementación de First Fit, Best Fit, Worst Fit y Buddy System
<code>ui/ganttRenderer.js</code>	Diagrama de Gantt sobre Canvas HTML5 (vistas compacta y por proceso)
<code>ui/memRenderer.js</code>	Mapa de memoria vertical dinámico con leyenda
<code>ui/metricsRenderer.js</code>	Cálculo y actualización de las diez métricas en tiempo real
<code>ui/stepLog.js</code>	Log narrativo de memoria y cola de procesos tick a tick

Nota. SPA = *Single Page Application*. Todos los módulos usan sintaxis ES6 `import/export` sin dependencias externas.

Flujo de la Simulación

Al pulsar “Ejecutar Simulación”, el orquestador invoca al planificador de CPU seleccionado. Este devuelve `eventosGantt` (lista de segmentos con proceso, inicio y fin), `procesosResultado`

(métricas por proceso) y `tiempoTotal`. Con esa información, el orquestador construye una línea de tiempo unificada y la reproduce tick a tick: en cada tick actualiza las diez métricas del dashboard, el diagrama de Gantt y, cuando un proceso inicia o finaliza, solicita al módulo de memoria la asignación o liberación del bloque correspondiente y refresca el mapa de memoria y el log.

Módulo de Planificación de CPU

FCFS. Ordena los procesos por tiempo de llegada y los ejecuta en secuencia. Si la CPU está ociosa al llegar el siguiente proceso, registra un evento IDLE. El código 1 muestra la implementación completa.

```
1 export function fcfs(procesos) {
2   if (!procesos.length)
3     return { eventosGantt: [], procesosResultado: [], tiempoTotal: 0
4       };
5
6   const lista = procesos.map(p => ({ ...p }));
7   lista.sort((a, b) => a.llegada - b.llegada);
8   const eventosGantt = [];
9   let tiempoActual = 0;
10
11   for (const proceso of lista) {
12     if (tiempoActual < proceso.llegada) {
13       eventosGantt.push({
14         proceso: 'IDLE', inicio: tiempoActual,
15         fin: proceso.llegada, color: '#d1cfc8'
16       });
17       tiempoActual = proceso.llegada;
18     }
19     const tiempoInicio = tiempoActual;
20     const tiempoFin = tiempoActual + proceso.ejecucion;
```

```

20     eventosGantt.push({
21         proceso: proceso.id, inicio: tiempoInicio,
22         fin: tiempoFin,         color: proceso.color
23     });
24     proceso.inicio           = tiempoInicio;
25     proceso.fin              = tiempoFin;
26     proceso.tiempoEspera     = tiempoInicio - proceso.llegada;
27     proceso.tiempoRetorno    = tiempoFin - proceso.llegada;
28     proceso.tiempoRespuesta = tiempoInicio - proceso.llegada;
29     tiempoActual = tiempoFin;
30 }
31 return { eventosGantt, procesosResultado: lista,
32         tiempoTotal: tiempoActual };
33 }

```

Listing 1

Implementación de FCFS – planificadores/algoritmos.js

SRT. Opera tick a tick. En cada unidad de tiempo selecciona el proceso de menor tiempo restante entre los disponibles. Cuando el proceso activo cambia, cierra el segmento anterior y abre uno nuevo. Al final fusiona segmentos consecutivos del mismo proceso para evitar barras fragmentadas. El código 2 muestra el núcleo.

```

1 while (completados < lista.length &&
2     tiempoActual <= tiempoMaximo) {
3     const disponibles = lista.filter(
4         p => !p.completado && p.llegada <= tiempoActual
5     );
6     let siguiente = disponibles.length > 0
7         ? disponibles.reduce((min, p) =>
8             p.tiempoRestante < min.tiempoRestante ? p : min)

```



```

9      : null;
10
11     if (siguiente !== procesoActual) {
12         if (procesoActual !== null)
13             eventosGantt.push({
14                 proceso: procesoActual.id,
15                 inicio: inicioSegmento, fin: tiempoActual,
16                 color: procesoActual.color
17             });
18         procesoActual = siguiente;
19         inicioSegmento = tiempoActual;
20         if (procesoActual && procesoActual.primerEjecucion === -1)
21             procesoActual.primerEjecucion =
22             procesoActual.inicio = tiempoActual;
23     }
24     if (procesoActual !== null) {
25         procesoActual.tiempoRestante--;
26         if (procesoActual.tiempoRestante === 0) {
27             procesoActual.fin = tiempoActual + 1;
28             procesoActual.tiempoRetorno = procesoActual.fin
29                                     - procesoActual.llegada;
30             procesoActual.tiempoEspera = procesoActual.tiempoRetorno
31                                     - procesoActual.ejecucion;
32             completados++;
33         }
34     }
35     tiempoActual++;
36 }

```

Listing 2

Núcleo del algoritmo SRT – planificadores/algoritmos.js

Módulo de Gestión de Memoria

La memoria se representa como un array ordenado de bloques. Cada bloque contiene id, tipo (libre, proceso o so), inicio, tamaño y color. La función compartida `asignarEnBloque()` divide el bloque candidato en dos: la parte asignada al proceso y el residuo libre. Al liberar, el sistema fusiona el bloque con sus vecinos libres adyacentes. El código 3 muestra la implementación de First Fit.

```
1 export function firstFit(proceso, memoria) {
2   for (let i = 0; i < memoria.length; i++) {
3     const bloque = memoria[i];
4     if (bloque.tipo !== 'libre') continue;
5     if (bloque.tamaño < proceso.tamañoKB) continue;
6     return asignarEnBloque(proceso, memoria, i);
7   }
8   return {
9     éxito: false,
10    descripcion:
11      'No hay hueco libre >= ${proceso.tamañoKB}KB. ' +
12      'Proceso ${proceso.id} rechazado.',
13    fragmentacionInterna: 0,
14  };
15 }
```

Listing 3

Implementación de First Fit – memoria/algoritmos.js

El Buddy System añade la lógica de potencias de 2. El código 4 muestra su función central.

```

1 export function buddySystem(proceso, memoria, pasos = []) {
2   const tamañoNec = siguientePotencia2(proceso.tamañoKB);
3   const candidatos = memoria
4     .map((b, i) => ({ ...b, indice: i }))
5     .filter(b => b.tipo === 'libre' && b.tamaño >= tamañoNec);
6
7   if (!candidatos.length)
8     return { éxito: false,
9             descripcion: 'Sin hueco >= ${tamañoNec}KB' };
10
11   let bloque = candidatos.reduce((min, b) =>
12     b.tamaño < min.tamaño ? b : min);
13
14   while (bloque.tamaño / 2 >= tamañoNec) {
15     const mitad = bloque.tamaño / 2;
16     bloque.tamaño = mitad;
17     memoria.splice(bloque.indice + 1, 0, {
18       tipo: 'libre', inicio: bloque.inicio + mitad,
19       tamaño: mitad, id: null
20     });
21     pasos.push('Dividido: 2 bloques de ${mitad}KB');
22   }
23   return asignarEnBloque(proceso, memoria, bloque.indice);
24 }

```

Listing 4

Asignación Buddy System – memoria/algoritmos.js

Dashboard de Métricas en Tiempo Real

El módulo `metricsRenderer.js` actualiza diez cajas del dashboard en cada tick de la simulación. Las cinco métricas de CPU se calculan en `actualizarMetricasCPU()` y las cinco de memoria en `actualizarMetricasMemoria()`. El código 5 muestra ambas funciones.

```
1 export function actualizarMetricasCPU(  
2   todosProcesos, tiempoActual, eventosGantt) {  
3   const completados = todosProcesos.filter(  
4     p => p.fin !== undefined && p.fin <= tiempoActual);  
5   // Tiempo de espera promedio  
6   const te = completados.length  
7     ? completados.reduce((s,p) => s + (p.tiempoEspera||0), 0)  
8     / completados.length  
9     : null;  
10  // Tiempo de retorno promedio  
11  const tr = completados.length  
12    ? completados.reduce((s,p) => s + (p.tiempoRetorno||0), 0)  
13    / completados.length  
14    : null;  
15  // Uso de CPU  
16  const tiempoUtil = eventosGantt  
17    .filter(ev => ev.proceso !== 'IDLE' &&  
18      ev.inicio < tiempoActual)  
19    .reduce((s, ev) =>  
20      s + (Math.min(ev.fin, tiempoActual) - ev.inicio), 0);  
21  const usoCPU = tiempoActual > 0  
22    ? (tiempoUtil / tiempoActual * 100).toFixed(1)  
23    : '0.0';  
24  // Throughput
```

```

25  const throughput = tiempoActual > 0
26    ? (completados.length / tiempoActual).toFixed(2)
27    : '0.00';
28  // Actualizar cajas
29  actualizarCajaMetrica('mvTotalProc', completados.length);
30  actualizarCajaMetrica('mvUsoCPU', usoCPU + '%');
31  actualizarCajaMetrica('mvTEPromedio', te ? te.toFixed(2):'--');
32  actualizarCajaMetrica('mvTRPromedio', tr ? tr.toFixed(2):'--');
33  actualizarCajaMetrica('mvThroughput', throughput);
34 }
35
36 export function actualizarMetricasMemoria(
37   metricas, totalProcesos) {
38   actualizarCajaMetrica(
39     'mvUsoMem', metricas.porcentajeUso + '%');
40   actualizarCajaMetrica(
41     'mvMemLibre', metricas.fragExterna + ' KB');
42   actualizarCajaMetrica(
43     'mvFragInt', metricas.fragInterna + 'KB');
44   actualizarCajaMetrica(
45     'mvFragExt', metricas.fragExterna + 'KB');
46   actualizarCajaMetrica(
47     'mvProcMem',
48     metricas.procesosAsignados + '/' + totalProcesos);
49 }

```

Listing 5

Funciones de métricas en tiempo real – ui/metricsRendererer.js

Interfaz de Usuario

La interfaz se organiza en tres secciones sin recarga de página. La sección **01 Configuración de Simulación** contiene la carga de procesos (arrastrar archivo o tabla editable) y el panel de selección de algoritmos con sus parámetros. La sección **02 Resultados y Visualización** muestra las diez cajas de métricas en tiempo real agrupadas en dos bloques (Memoria y CPU), los controles de animación (paso a paso, automático, velocidad), el mapa de memoria vertical, el diagrama de Gantt (vistas compacta y por proceso) y el panel dividido entre el log de pasos de memoria y la cola de procesos tick a tick. La sección **03 Resultados Detallados** presenta la tabla completa de planificación de CPU con nueve columnas y la tabla de gestión de memoria con siete columnas, incluyendo el análisis de fragmentación.

RESULTADOS

Las pruebas se ejecutaron con el archivo `procesos_5.txt` (cinco procesos), la configuración FCFS + First Fit, 400 KB de memoria total, 0 KB reservados para el SO y política dinámica.

La Tabla 2 resume la carga de trabajo.

Tabla 2

Conjunto de procesos de prueba (`procesos_5.txt`)

Proceso	Llegada (t)	Ejecución (t)	Tamaño (KB)
P1	0	5	50
P2	1	3	5
P3	2	8	80
P4	3	2	15
P5	4	5	50

Nota. Fuente: archivo `procesos_5.txt` incluido en el repositorio del proyecto.

Configuración de la Simulación

La Figura 1 muestra la sección 01 tras importar el archivo. La zona de carga confirma cinco procesos leídos y la tabla editable refleja los valores inmediatamente. La Figura 2 muestra el panel de algoritmos: FCFS seleccionado (no expulsivo, orden de llegada) y First Fit como estrategia de memoria (primer hueco suficiente), con 400 KB totales y política dinámica.

5.0

Simulador del Núcleo S.O.

CPU: -

MEM: -

01

Configuración de Simulación

Define los procesos y selecciona los algoritmos de planificación

Carga de Trabajo

ARCHIVO DE PROCESOS (.CSV / .TXT)

Arrastra tu archivo aquí o haz clic

proceso, llegada, ejecución, tamaño_KB

procesos_5.txt (5 proceso(s))

FORMATO DEL ARCHIVO

proceso, llegada, ejecución, tamaño_KB

P1, 0, 5, 50

P2, 1, 3, 30

Líneas con # son comentarios

PROCESOS (EDICIÓN MANUAL)

PROCESO	LLEGADA	EJECUCIÓN	TAMAÑO (KB)	
P1	0	5	50	×
P2	1	3	5	×
P3	2	8	80	×
P4	3	2	15	×
P5	4	5	50	×

+ Agregar proceso

Figura 1
Sección 01 – Carga de procesos con *procesos_5.txt* importado

Configuración de Algoritmos

PLANIFICADOR DE CPU

FCFS

First Come First Served - No expulsivo

NO EXPULSIVO

SJF

Shortest Job First - Menor ráfaga primero

NO EXPULSIVO

RR

Round Robin - Quantum configurable

EXPULSIVO

SRT

Shortest Remaining Time - SJF con expulsión

EXPULSIVO

Quantum (RR)

- 2 +

ALGORITMO DE MEMORIA

First Fit

Primer Ajuste - Primer hueco suficiente

RÁPIDO

Best Fit

Mejor Ajuste - Hueco más pequeño suficiente

PRECISO

Worst Fit

Peor Ajuste - Hueco más grande disponible

HEURÍSTICO

Buddy System

Potencias de 2 - Fusión de gemelos

POTENCIAS 2

MEMORIA TOTAL (KB)

400

SO RESERVADO (KB)

0

POLÍTICA DE PARTICIÓN

Dinámica

Fija

Figura 2
Panel de configuración de algoritmos: *FCFS + First Fit, 400 KB, política dinámica*

Dashboard de Métricas al Finalizar

La Figura 3 muestra el dashboard en el paso 10/10 ($t=23$), con la simulación completamente terminada. La Tabla 3 transcribe los diez valores mostrados.

15

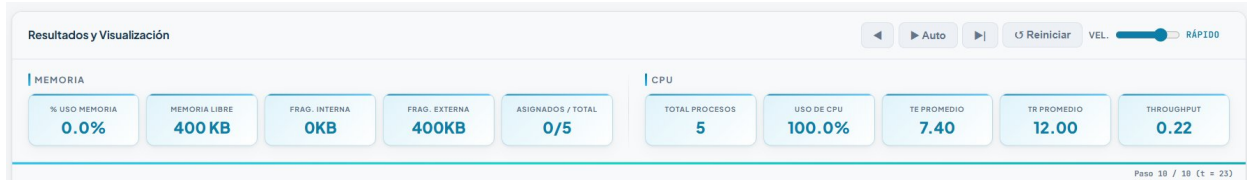


Figura 3
Dashboard de diez métricas al finalizar la simulación (paso 10/10, $t=23$)

Tabla 3
Valores del dashboard al finalizar la simulación ($t = 23$)

Bloque	Métrica	Valor
Memoria	% Uso de Memoria	0.0 %
	Memoria Libre	400 KB
	Fragmentación Interna	0 KB
	Fragmentación Externa	400 KB
	Asignados / Total	0 / 5
CPU	Total Procesos	5
	Uso de CPU	100.0 %
	T.E. Promedio	7.40
	T.R. Promedio	12.00
	Throughput	0.22

Nota. Los valores de memoria corresponden al estado final tras la liberación de todos los procesos. T.E. = Tiempo de Espera; T.R. = Tiempo de Retorno. Unidades de tiempo abstractas (u). Throughput en procesos/u.

Los valores de memoria (0.0 % de uso, 400 KB libres, 0/5 asignados) reflejan el estado *post-liberación*: todos los procesos terminaron y liberaron sus bloques antes del instante final $t=23$. La fragmentación externa de 400 KB equivale a la totalidad del espacio disponible reconstituido como un único hueco continuo, confirmando que el mecanismo de fusión de bloques libres adyacentes funciona correctamente. En el lado de CPU, el uso del 100 % confirma que no existieron períodos ociosos durante los 23 ticks de simulación.

Mapa de Memoria en Instante Intermedio

La Figura 4 muestra el mapa de memoria en el instante en que P1, P2 y P3 ya fueron liberados pero P4 y P5 permanecen activos. El espacio 0–134 KB (135 KB) quedó libre tras las liberaciones, P4 ocupa 135–149 KB y P5 ocupa 150–199 KB, con un bloque libre inferior de 200 KB (200–399 KB). El mapa totaliza cuatro bloques para 400 KB, lo que confirma la fusión correcta de los tres bloques liberados consecutivos.



Figura 4

Mapa de memoria con P4 y P5 activos tras liberación de P1, P2 y P3 (instante intermedio)

Diagrama de Gantt Completo

La Figura 5 muestra el diagrama de Gantt en vista compacta al finalizar la simulación ($T=23$, 5 eventos). Los cinco procesos se ejecutan de forma secuencial sin solapamiento ni períodos ociosos: P1 en $t=0-5$, P2 en $t=5-8$, P3 en $t=8-16$, P4 en $t=16-18$ y P5 en $t=18-23$. La ausencia de huecos confirma que la CPU trabajó al 100 % durante toda la simulación.

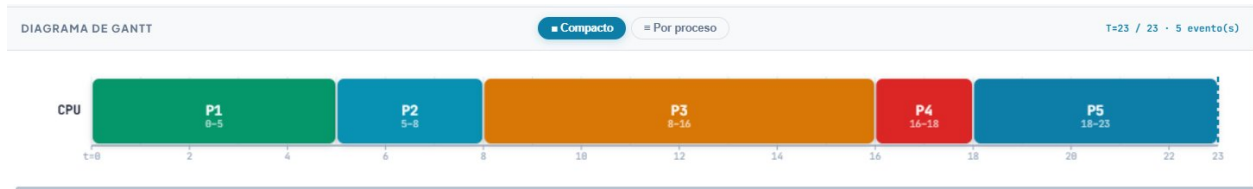


Figura 5

Diagrama de Gantt compacto al finalizar la simulación – FCFS, $T=23$, sin períodos ociosos

Log de Pasos y Cola de Procesos

La Figura 6 muestra el panel inferior sincronizado de la sección 02. El log de pasos (izquierda, 10 pasos) registra las cinco asignaciones de memoria y las cinco liberaciones con sus direcciones exactas y la fragmentación interna residual en cada instante. La cola de procesos (derecha, 11 ticks) detalla el estado de la CPU y la lista de espera tick a tick: en $t=0$ la CPU arranca con IDLE y en $t=0$ pasa a ejecutar P1, en $t=1$ P2 ingresa a la cola mientras P1 continúa, y en $t=2$ tanto P2 como P3 aguardan la CPU.

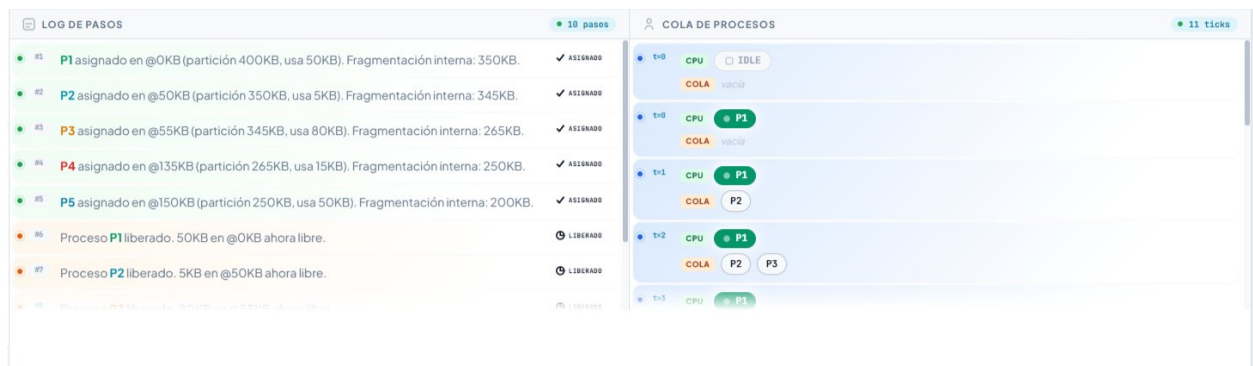


Figura 6

Log de pasos de memoria (10 pasos) y cola de procesos CPU (11 ticks) sincronizados

Resultados Detallados – Tablas Finales

La Figura 7 presenta la sección 03 completa con las dos tablas al finalizar la simulación. La barra de encabezado confirma la configuración usada: CPU: FCFS y MEM: First Fit.

0.0 Simulador del Núcleo S.O. CPU: FCFS MEM: First Fit

03 Resultados Detallados
Métricas completas por proceso

Tabla de Planificación de CPU

PROCESO	LLEGADA	EJECUCIÓN	INICIO	FIN	T. ESPERA	T. RETORNO	T. RESPUESTA	ESTADO
P1	0	5	0	5	0	5	0	✓ Completado
P2	1	3	5	8	4	7	4	✓ Completado
P3	2	8	8	16	6	14	6	✓ Completado
P4	3	2	16	18	13	15	13	✓ Completado
P5	4	5	18	23	14	19	14	✓ Completado

Tabla de Gestión de Memoria

PROCESO	TAMAÑO SOL.	PARTICIÓN (KB)	DIR. INICIO	DIR. FIN	FRAG. EXTERNA	ESTADO
P1	50 KB	400 KB	0 KB	49 KB	350 KB	tr. Liberado
P2	5 KB	350 KB	50 KB	54 KB	345 KB	tr. Liberado
P3	80 KB	345 KB	55 KB	134 KB	245 KB	tr. Liberado
P4	15 KB	265 KB	135 KB	149 KB	250 KB	tr. Liberado
P5	50 KB	250 KB	150 KB	199 KB	200 KB	tr. Liberado

Figura 7

Sección 03 – Tabla de planificación de CPU y tabla de gestión de memoria (FCFS + First Fit)

La Tabla 4 transcribe los valores de planificación de CPU. P1 no espera (llegó con la CPU libre). P4 y P5 acumulan los mayores tiempos de espera (13 u y 14 u) por el efecto convoy de FCFS.

Tabla 4*Resultados de planificación de CPU – FCFS, cinco procesos*

P.	Lleg.	Ejec.	Inicio	Fin	T.E.	T.R.	T.Resp.	Estado
P1	0	5	0	5	0	5	0	Completado
P2	1	3	5	8	4	7	4	Completado
P3	2	8	8	16	6	14	6	Completado
P4	3	2	16	18	13	15	13	Completado
P5	4	5	18	23	14	19	14	Completado
Prom.	–	–	–	–	7.40	12.00	7.40	–

Nota. P. = Proceso; Lleg. = Llegada; Ejec. = Ejecución; T.E. = Tiempo de Espera; T.R. = Tiempo de Retorno; T.Resp. = Tiempo de Respuesta. Valores en unidades de tiempo abstractas (u).

La Tabla 5 transcribe los resultados de gestión de memoria. La columna “Frag. Externa” muestra el espacio libre residual del bloque en el momento de cada asignación (valor decreciente conforme se asignan procesos). El estado final de todos los procesos es “Liberado”, consistente con el dashboard que reporta 0/5 asignados y 400 KB libres al terminar.

Tabla 5*Resultados de gestión de memoria – First Fit, 400 KB dinámico*

Proc.	Sol.(KB)	Part.(KB)	Dir. Inicio	Dir. Fin	Frag. Ext.	Estado
P1	50	400	0 KB	49 KB	350 KB	Liberado
P2	5	350	50 KB	54 KB	345 KB	Liberado
P3	80	345	55 KB	134 KB	265 KB	Liberado
P4	15	265	135 KB	149 KB	250 KB	Liberado
P5	50	250	150 KB	199 KB	200 KB	Liberado
Fragmentación interna total					0 KB	
Fragmentación externa final					400 KB	

Nota. “Part.(KB)” es el tamaño del bloque libre en el momento de la asignación. La columna “Frag. Ext.” es el espacio libre residual del bloque tras asignar al proceso, no un desperdicio dentro de la partición entregada. Con política dinámica la fragmentación interna real es 0 KB. El estado “Liberado” confirma que todos los procesos devolvieron su memoria al finalizar.

CONCLUSIONES

El Simulador del Núcleo S.O. cumplió todos los objetivos planteados. Los ocho algoritmos producen resultados coherentes con los valores teóricos, y el dashboard de diez métricas actualiza correctamente los cinco indicadores de CPU y los cinco de memoria en tiempo real durante la simulación.

Con FCFS y First Fit, el conjunto de prueba ilustra con claridad el efecto convoy: P4 y P5 acumulan tiempos de espera de 13 y 14 unidades respectivamente a pesar de tener ráfagas de 2 y 5 unidades, porque los tres procesos previos los bloquean en la cola. El uso de CPU del 100 % confirma la ausencia de períodos ociosos con esta carga de trabajo particular.

En la gestión de memoria, la política dinámica con First Fit asignó los cinco procesos sin rechazos y sin fragmentación interna. La fragmentación externa de 400 KB al finalizar refleja la totalidad del espacio disponible, reconstituido en un único bloque continuo gracias al mecanismo de fusión de vecinos libres.

Como trabajo futuro se propone incorporar la estimación de ráfagas por envejecimiento, un módulo de sincronización con semáforos y la posibilidad de exportar los resultados en formato PDF o CSV directamente desde la interfaz.

REFERENCIAS

- Knuth, D. E. (1973). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (2.^a ed.). Addison-Wesley.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10.^a ed.). Wiley.
- Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9.^a ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4.^a ed.). Pearson.

ANEXO

Repositorio de Código Fuente en GitHub

El código fuente completo del Simulador del Núcleo S.O. está disponible públicamente en el siguiente repositorio:

<https://github.com/Abad-cyber/Sistema-del-nucleo-S.O..git>

El repositorio incluye la estructura modular completa descrita en la Tabla 1: los módulos de planificación de CPU, los de gestión de memoria, los cuatro renderizadores de interfaz de usuario, el orquestador principal y el archivo de procesos de prueba `procesos_5.txt`. Para ejecutar el simulador localmente basta con clonar el repositorio y abrir `index.html` en un navegador moderno compatible con módulos ES6.